

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:100

Annexure A

CASE STUDY

Code of Conduct:

I D K SUMALATHA(192424023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D K SUMALATHA

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

CASE STUDY

Set 1

Q1. Both parse trees and syntax trees are used to represent program structure, but differ significantly in complexity and abstraction level. Compare these representations in terms of memory usage, ease of traversal, suitability for optimization, and their role in different phases of compilation such as semantic analysis and code generation. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, representation choice affects compiler performance. Finally, selecting the right abstraction improves overall efficiency.

Compare Parse Trees and Syntax Trees

A **parse tree** and a **syntax tree (AST – Abstract Syntax Tree)** both represent the structure of a program, but they differ in their level of detail and purpose.

Aspect	Parse Tree	Syntax Tree / AST
Representation	Shows complete grammar structure	Shows only essential program structure
Memory usage	Higher because all grammar symbols are stored	Lower because unnecessary symbols are removed
Traversal	More difficult because it contains many nodes	Easier and faster because it contains fewer nodes
Optimization	Less suitable for optimization	Highly suitable for optimization
Semantic analysis	Can be used to verify grammatical structure	More commonly used for type checking and semantic analysis

Code generation	Less efficient for direct code generation	Very suitable for intermediate and target code generation
------------------------	---	---

For example, consider:

$a + b * c$

The parse tree contains grammar symbols such as Expression, Term, and Factor. The syntax tree mainly contains the important operators and operands:

```

      +
     /\
    a  *
     /\
    b  c

```

Attribute Dependency

During semantic analysis, **attributes** are attached to grammar symbols. A dependency graph determines which attributes must be calculated first.

- **Synthesized attributes** generally move information from child to parent.
- **Inherited attributes** pass information from parent or siblings to a child.

If dependencies are not evaluated in the correct order, incorrect semantic results may occur. Therefore, the compiler must ensure that the dependency graph does not contain conflicting or circular dependencies.

Role in Compilation

- **Parse trees** are mainly useful during syntax analysis.
- **Syntax trees** are more useful during semantic analysis, intermediate code generation, optimization, and final code generation.
- ASTs reduce memory requirements and improve compiler performance because unnecessary grammar information is removed.

Conclusion

Therefore, **parse trees provide detailed grammatical information**, while **syntax trees provide a compact and efficient representation of program meaning**. For modern compilers, syntax trees are generally preferred for semantic analysis, optimization, and code generation because they require less memory and are easier to traverse.

Set 2

1. A compiler development team is designing a new programming language for scientific applications. During semantic analysis, the compiler must evaluate arithmetic expressions involving multiple operators and nested function calls. The team plans to use Syntax-Directed Definitions (SDD) to associate semantic rules with grammar productions. They also need to construct syntax trees for efficient intermediate code generation. However, incorrect attribute evaluation is causing inconsistent outputs during parsing. Analyze how synthesized and inherited attributes can be used to solve this issue and justify whether bottom-up or top-down evaluation is more suitable for the given scenario.

Q1. Synthesized and Inherited Attributes in SDD

Syntax-Directed Definition (SDD) associates semantic rules with grammar productions. These rules help the compiler calculate attributes and construct syntax trees.

Consider the expression:

$$a + b * c$$

The compiler can construct the syntax tree while applying semantic rules.

Synthesized Attributes

A **synthesized attribute** obtains its value from the attributes of child nodes.

For example:

$$E \rightarrow E1 + T$$
$$E.val = E1.val + T.val$$

The value of E is calculated from its children.

Synthesized attributes are especially suitable for:

- Arithmetic expressions
- Expression evaluation
- Syntax-tree construction
- Intermediate-code generation

They naturally support **bottom-up evaluation**.

Inherited Attributes

An **inherited attribute** obtains information from the parent or sibling nodes.

For example, when processing:

function(a, b)

information such as the function's expected parameter type can be passed from the function declaration to individual parameters.

Inherited attributes are useful for:

- Function parameters
- Variable declarations
- Scope information
- Type information
- Context-dependent semantic checking

Solving the Incorrect Evaluation Problem

The compiler should create an **attribute dependency graph**. This graph identifies which attributes depend on other attributes.

For example:

Child attributes



Parent synthesized attribute



Syntax tree / intermediate representation

For contextual information:

Parent information



Inherited attribute



Child node

The compiler should evaluate each attribute only after all its required dependencies are available. This prevents inconsistent outputs.

Bottom-Up or Top-Down?

For arithmetic expressions and syntax-tree construction, **bottom-up evaluation is more suitable** because synthesized attributes can be calculated naturally after processing the operands.

However, if the language contains many inherited attributes for function parameters and scopes, **top-down/L-attributed evaluation** becomes useful.

Conclusion

For the given scientific language, **bottom-up evaluation with synthesized attributes should be the primary approach for arithmetic expressions**, while inherited attributes should be introduced where contextual information such as function parameters and scopes is required. This combination provides reliable semantic evaluation and efficient intermediate-code generation.

Set 3

A software company is developing a mini programming language for scientific calculations, where arithmetic expressions must be translated into syntax trees before code generation. During compilation, the compiler team notices that inherited and synthesized attributes are not being evaluated correctly for nested expressions. Initially, they use S-attributed definitions for bottom-up evaluation but later require L-attributed definitions to support function parameters and variable scopes. The compiler must also perform semantic checks to ensure that integer and floating-point operations are compatible. To improve reliability, the team plans to integrate automatic type conversion wherever necessary.

a) Explain how syntax-directed definitions help in syntax tree construction.

b) Compare S-attributed and L-attributed definitions in this scenario.

a) How do Syntax-Directed Definitions help in syntax tree construction?

A **Syntax-Directed Definition (SDD)** attaches semantic rules to grammar productions. These rules specify what information should be calculated whenever a grammar production is recognized.

For example:

$$E \rightarrow E1 + T$$

A semantic rule can create an operator node:

$$E.\text{node} = \text{newNode}('+', E1.\text{node}, T.\text{node})$$

For:

$$a + b * c$$

the resulting syntax tree is:

$+$
 $/\backslash$
 $a \quad *$
 $/\backslash$
 $b \quad c$

SDDs therefore help the compiler:

1. Recognize grammatical structures.
2. Create syntax-tree nodes.
3. Attach attributes to nodes.
4. Perform semantic analysis.
5. Generate intermediate code.

This provides a direct connection between **syntax analysis and semantic processing**.

b) Compare S-Attributed and L-Attributed Definitions

Feature	S-Attributed Definition	L-Attributed Definition
Attributes	Only synthesized	Synthesized + restricted inherited
Information flow	Child $\rightarrow$ parent	Parent/sibling $\rightarrow$ child and child $\rightarrow$ parent
Evaluation	Bottom-up	Usually left-to-right/top-down compatible
Implementation	Simple	More flexible
Suitable for	Arithmetic expressions	Functions, scopes, declarations
Case suitability	Simple expressions	Function parameters and variable scopes

S-Attributed Definition

S-attributed definitions use only synthesized attributes.

Example:

$$E \rightarrow E1 + T$$
$$E.val = E1.val + T.val$$

This works naturally with **bottom-up parsing**.

L-Attributed Definition

L-attributed definitions allow inherited attributes, provided their dependencies follow a left-to-right evaluation order.

They are useful when information needs to be passed from a parent or previous sibling.

For example, a function declaration may provide a type to its parameters:

```
int function(int a, int b)
```

The parameter type can be inherited by the parameter nodes.

Type Checking and Conversion

The compiler should check whether operations such as:

```
int + float
```

are valid. If they are compatible, the compiler can automatically convert:

```
int → float
```

before generating intermediate code.

Conclusion

S-attributed definitions are simple and efficient for bottom-up expression processing, while L-attributed definitions are more suitable when contextual information such as function parameters and scopes is required. Therefore, the language should use S-attributed definitions for simple expressions and L-attributed definitions for more complex semantic contexts.

Set 4

1.

A game development company is creating a custom scripting language for controlling character movements and animations. The compiler team uses syntax-directed definitions to generate syntax trees for arithmetic and logical expressions during parsing. Initially, bottom-up evaluation with S-attributed definitions works well for simple expressions, but difficulties arise when inherited attributes are needed for function calls and nested scopes. During semantic analysis, the compiler detects several type mismatches between integer, float, and boolean variables. To ensure smooth execution, the developers implement a type checker with automatic type conversion support.

a) How do syntax-directed definitions assist in syntax tree construction?

b) Why are type conversions necessary in semantic analysis?

a) How do Syntax-Directed Definitions assist in syntax tree construction?

Syntax-Directed Definitions attach semantic actions to grammar productions. When the parser recognizes a production, the corresponding semantic rule is executed.

For example:

$$E \rightarrow E1 + E2$$

The semantic action can create a + node:

```
E.node = newNode('+', E1.node, E2.node)
```

For:

$x + y * z$

the compiler creates:

+

/ \

x *

/ \

y z

This syntax tree can then be used for:

- Semantic analysis
- Type checking
- Intermediate-code generation
- Optimization
- Final code generation

Thus, SDD provides a systematic connection between grammar recognition and syntax-tree construction.

b) Why are Type Conversions Necessary in Semantic Analysis?

Type conversion is necessary when different compatible data types are used in the same expression.

For example:

int a = 10;

float b = 5.5;

a + b

Here, int and float are different types. The compiler can convert:

int $\rightarrow$ float

and perform:

$10.0 + 5.5$

Importance of Type Conversion

1. Prevents invalid operations.
2. Maintains datatype compatibility.
3. Avoids unnecessary compilation errors.
4. Allows expressions containing compatible types.
5. Improves program reliability.
6. Ensures correct intermediate-code generation.

The compiler should distinguish between:

- **Safe/implicit conversion** – automatically performed.
- **Explicit conversion** – requested by the programmer.

For example:

$\text{int} + \text{float} \rightarrow \text{float}$

is generally acceptable, while:

$\text{int} + \text{boolean}$

should normally produce a type error.

Conclusion

Type checking detects incompatible operations, while type conversion makes compatible operations possible. Together, they improve the semantic correctness and reliability of the scripting language.

Set 5

1. A compiler design team is developing a semantic analyzer for a newly proposed programming language used in banking applications. The compiler must process arithmetic expressions, conditional statements, and nested loops while maintaining datatype consistency. The team decides to use Syntax-Directed Definitions to attach semantic rules with grammar productions and construct syntax trees for efficient translation. During testing, inherited attributes are not correctly propagated in nested expressions, leading to incorrect semantic outputs. Analyze how L-attributed definitions and top-down translation techniques can resolve these semantic dependency issues and improve compiler accuracy.

L-Attributed Definitions and Top-Down Translation

The banking compiler needs to process arithmetic expressions, conditional statements, and nested loops while maintaining datatype consistency. The problem occurs because inherited attributes are not correctly propagated through nested expressions.

L-Attributed Definitions

An **L-attributed definition** allows both synthesized and inherited attributes, with the restriction that inherited attributes of a symbol depend only on:

- The attributes of its parent.
- The attributes of symbols to its left.
- Previously evaluated information.

This makes the evaluation order predictable.

For example, consider:

int a, b, c;

The datatype int can be passed as an inherited attribute to each identifier:

Declaration

```

/   \
int   Identifier
    |
    a

```

The type information can then be propagated to a.

Top-Down Translation

Top-down translation processes the input from the parent toward its children.

This is useful when information must be passed downward.

For example:

Parent scope

↓

Child statement

↓

Nested block

↓

Variable

A symbol table or scope information can therefore be passed through inherited attributes.

Why It Solves the Problem

L-attributed definitions ensure that:

1. Dependencies are evaluated in a valid order.
2. Parent information reaches nested expressions.
3. Scope information can be propagated.

4. Type information reaches identifiers.
5. Function and loop contexts can be maintained.
6. Circular dependencies can be avoided.

Example

Suppose:

```
int x;  
  
{  
  
    x = 10;  
  
}
```

The declaration establishes:

$x \rightarrow \text{int}$

When the nested block uses x , the semantic analyzer can access the surrounding scope and verify that the assignment is valid.

Conclusion

For this banking language, **L-attributed definitions combined with top-down translation are highly suitable** because nested loops, conditional statements, variable scopes, and inherited contextual information require controlled information flow. This improves semantic accuracy and prevents incorrect attribute evaluation.

Set 6

1

A healthcare software company is developing a medical data processing language where patient records and calculations must be translated accurately into

intermediate code. The compiler initially uses syntax-directed definitions with synthesized attributes to construct syntax trees for arithmetic expressions. However, when handling nested function calls and variable dependencies, inherited attributes become necessary, leading the team to adopt L-attributed definitions. During semantic analysis, type mismatch errors frequently occur between integer, float, and string data used in calculations. To improve reliability, the compiler team introduces a type checker with type equivalence verification and automatic type conversion mechanisms.

a) Explain how syntax-directed definitions support semantic analysis.

b) Why are type checking and type conversion important in this scenario?

a) How do Syntax-Directed Definitions support Semantic Analysis?

Syntax-Directed Definitions associate semantic rules with grammar productions. They allow the compiler to perform semantic operations while processing the syntax.

For example:

$$E \rightarrow E1 + E2$$

The semantic rules can:

- Construct syntax-tree nodes.
- Calculate expression types.
- Check operand compatibility.
- Generate intermediate code.
- Maintain symbol-table information.

For the medical data language, an expression such as:

age + temperature

can be represented using a syntax tree and then checked semantically.

When nested function calls occur, inherited attributes can provide information such as:

- Function parameter types.
- Return types.
- Scope information.
- Expected expression type.

L-attributed definitions are useful because this information can be propagated in an organized manner.

b) Importance of Type Checking and Type Conversion

Medical data processing requires high reliability because incorrect calculations can lead to incorrect results.

Consider:

`int age = 25;`

`float temperature = 37.5;`

An expression:

`age + temperature`

requires compatible types.

The compiler can perform:

`int → float`

and calculate:

`25.0 + 37.5`

Type Checking

Type checking verifies whether an operation is valid.

Examples:

`int + float     → valid`

`float + int     → valid`

string + string → language-dependent

int + string → invalid

Type Equivalence

The compiler must determine whether two types are compatible.

For example:

int

float

string

are different types and should not be treated as automatically equivalent.

Type Conversion

When types are compatible, conversion can be performed.

Example:

int → float

This is often called **widening conversion**.

Importance

Type checking and conversion:

- Detect semantic errors.
- Prevent invalid calculations.
- Maintain data consistency.
- Support mixed-type expressions.
- Improve reliability.
- Generate correct intermediate code.

Conclusion

SDDs provide the framework for semantic analysis, while **type checking ensures correctness and type conversion allows compatible data types to work together safely**. This is especially important in a medical data processing language where accuracy is critical.

Set 7

1

A mobile application company is developing a lightweight programming language for smart devices where expressions and assignments must be translated efficiently into intermediate code. The compiler team initially applies syntax-directed definitions with synthesized attributes to build syntax trees for arithmetic operations. As the language expands with nested conditional statements and function calls, inherited attributes become necessary, leading to the use of L-attributed definitions. During testing, semantic errors occur because variables of incompatible types are used together in expressions and assignments. To improve compilation accuracy, the developers introduce a type checker capable of verifying type equivalence and performing safe type conversions.

a) Explain the role of syntax-directed definitions in syntax tree construction.

b) How do type checking and type conversion improve semantic correctness?

a) Role of Syntax-Directed Definitions in Syntax Tree Construction

Syntax-Directed Definitions connect grammar productions with semantic rules. They allow the compiler to create syntax-tree nodes while parsing expressions and assignments.

For example:

$E \rightarrow E_1 + E_2$

can generate:

+

/\

E1 E2

For:

$a + b * c$

the syntax tree becomes:

+

/\

a *

/\

b c

The tree represents the actual structure of the expression and can be used for:

- Semantic analysis.
- Type checking.
- Intermediate-code generation.
- Optimization.
- Code generation.

When nested conditions and function calls are introduced, inherited attributes can provide contextual information. Therefore, L-attributed definitions become useful.

b) How do Type Checking and Type Conversion Improve Semantic Correctness?

Type checking verifies that variables and expressions are used with compatible types.

For example:

int a;

float b;

$a + b;$

The compiler can safely convert:

$a \rightarrow \text{float}$

and perform the operation.

However:

`int a;`

`boolean b;`

$a + b;$

should normally generate a semantic error because the types are incompatible.

Benefits

1. Detects type errors

It identifies invalid assignments and operations.

2. Maintains type consistency

Variables are used according to their declared types.

3. Supports compatible mixed expressions

For example:

$\text{int} + \text{float} \rightarrow \text{float}$

4. Prevents incorrect code generation

Invalid expressions are rejected before machine code is generated.

5. Improves reliability

The final program behaves according to the language's type rules.

Conclusion

Type checking determines **whether an operation is valid**, while type conversion determines **how compatible types can be safely used together**. Therefore, both are essential for semantic correctness in the mobile programming language.

Set 8

1. A cloud computing company is developing a compiler for a distributed programming language that supports complex arithmetic and logical expressions. The compiler must construct syntax trees for efficient intermediate representation and perform semantic analysis using Syntax-Directed Definitions. During testing, incorrect attribute propagation causes errors in nested conditional statements and function calls. The compiler team decides to redesign the semantic rules using inherited and synthesized attributes. Analyze how S-attributed and L-attributed definitions can improve semantic evaluation and discuss which parsing strategy is more suitable for this compiler design.

S-Attributed and L-Attributed Definitions

The distributed programming language supports complex arithmetic and logical expressions, nested conditional statements, and function calls. Incorrect attribute propagation is causing semantic errors.

S-Attributed Definitions

S-attributed definitions use only **synthesized attributes**.

Information flows from children to their parent:

Parent

↑

Child Child

For example:

$E \rightarrow E1 + T$

$E.val = E1.val + T.val$

They are particularly suitable for:

- Arithmetic expressions.
- Syntax-tree construction.
- Bottom-up parsing.
- Intermediate-code generation.

Therefore, S-attributed definitions provide a simple and efficient approach for basic expressions.

L-Attributed Definitions

L-attributed definitions support synthesized attributes and restricted inherited attributes.

Information can flow:

Parent

↓

Child

and from previously processed siblings:

Left sibling → Right sibling

They are useful for:

- Nested conditional statements.
- Function parameters.
- Variable scopes.
- Type information.
- Context-dependent semantic analysis.

For example, a function's parameter type can be passed to its parameter nodes.

Comparison

Feature	S-Attributed	L-Attributed
Attributes	Synthesized only	Synthesized + inherited
Information flow	Bottom-up	Left-to-right with inherited information
Evaluation	Bottom-up	Top-down/left-to-right compatible
Complexity	Simple	More flexible
Arithmetic expressions	Excellent	Excellent
Nested scopes	Limited	Suitable
Function parameters	Limited	Suitable
Conditional contexts	Limited	Suitable

Suitable Parsing Strategy

For simple arithmetic expressions, **bottom-up parsing with S-attributed definitions** is highly suitable because synthesized attributes can be calculated after the child expressions are processed.

However, this compiler contains:

- Nested conditional statements.
- Function calls.
- Context-dependent semantic information.
- Variable dependencies.

Therefore, **L-attributed definitions with a top-down or left-to-right compatible translation strategy** are more suitable for the complete compiler.

Attribute Dependency Graph

The compiler should construct an attribute dependency graph to determine the correct evaluation order:

Parent information

↓

Inherited attribute

↓

Child processing

↓

Synthesized attribute

↓

Parent

The compiler must ensure that every attribute is evaluated only after its dependencies are available and that circular dependencies do not occur.

Conclusion

S-attributed definitions are best for simple bottom-up expression processing, while L-attributed definitions are better for the complete distributed language because inherited information is required for nested conditions and function calls. Therefore, the recommended design is to use S-attributed processing wherever possible and L-attributed processing where contextual information is required. This gives a good balance between **simplicity, correctness, memory efficiency, and compiler performance.**